

학습 노트 (3). 스프라이트와 점수판, 그리고 키보드 입력의 시간 문제

공과대학 기계공학부 안준영 학번 2024-11078 2026년 10월 8일

들어가며

이번 주 과제는 지난주 학습 사항의 미진한 부분을 보완하는 네 가지다. Q.1과 Q.4의 베릴로그 RTL에서 그래픽 장치를 다루는 방식의 차이를 설명하는 일, Q.10을 실습하고 스프라이트 구현 방식을 아두이노 C++와 베릴로그 RTL 사이에서 비교하는 일, Q.11을 실습하고 좌우 화살표 키 입력의 이상 반응 원인을 찾는 일, 그리고 Q.13의 시뮬레이션 디렉토리에서 실습하고 테스트벤치와 RTL의 변경 사항을 찾는 일이다. 네 가지를 모두 수행했고, 그 과정에서 발견한 코드상의 오류도 함께 정리했다.

과제 1. Q.1과 Q.4에서 그래픽 장치를 다루는 방식의 차이

두 코드의 포트 목록을 나란히 놓으면 차이가 바로 보인다. Q.1의 `space_invaders_engine.v`에서는 화면 좌표 `vga_x`와 `vga_y`, 그리고 `video_on`이 모두 입력이다. 엔진 밖의 VGA 타이밍 회로가 640x480 화면을 주사하면서 현재 픽셀 좌표를 엔진에 건네주면, 엔진은 그 좌표가 대포나 포탄이나 외계인의 영역에 속하는지 조합 논리로 판정하고 rgb 출력을 돌려줄 뿐이다. 화면을 훑는 주도권은 디스플레이 쪽에 있고, 엔진은 질문에 답하는 수동적인 위치다. 화면 메모리 없이 픽셀이 지나가는 순간에 색을 즉석에서 계산하는 방식이다.

Q.4 이후의 `space_invaders_engine_glcd.v`에서는 `lcd_x`, `lcd_y`, `lcd_write_en`, `lcd_data`가 모두 출력이다. 엔진이 `scan_x`와 `scan_y` 카운터를 내부에 두고 128x64 전체를 스스로 주사하면서, 매 클럭 한 픽셀씩 가상 LCD의 그래픽 메모리에 써 넣는다. 해상도는 640x480에서 128x64로, 색은 3비트 RGB에서 1비트 단색으로 바뀌었지만 더 본질적인 변화는 주도권이다. 엔진이 디스플레이 메모리에 직접 쓰는 버스 마스터가 됐다. 같은 게임을 만들면서도 그래픽 장치를 대하는 관점이 외부 비디오 신호에 종속된 픽셀 응답기에서, 메모리에 매핑된 장치를 구동하는 능동적인 송신기로 완전히 바뀐 것이다. 입출력의 방향이 뒤집혔다는 사실 하나가 이 설계 사상의 차이를 그대로 담고 있다.

과제 2. Q.10 실습과 스프라이트 구현 방식 비교

Q.10 디렉토리에서 빌드하고 실행했다. 시작 대기 상태에서는 지난주와 같은 체크무늬가 나오고, 스페이스바를 누르면 게임이 시작된다. Fig. 1은 빌드 성공 직후의 시작 화면이다.

```

junyoung2@junyoung2: ~/ETRI050_DesignKit
g++ -I. -MMO -I/usr/share/verilator/include -I/usr/share/verilator/include/vl
tstd -DVM_COVERAGE=0 -DVM_SC=1 -DVM_TRACE=0 -faligned-new -fcf-protection=none
-Wno-bool-operation -Wno-sign-compare -Wno-uninitialized -Wno-unused-but-set-va
riable -Wno-unused-parameter -Wno-unused-variable -Wno-shadow -std=c++17 -I
/usr/include/SDL2 -D_REENTRANT -I/opt/systemc/include -I../Q.09b_Import_into
_SystemC_All -I/opt/systemc/include -Os -c -o Vspace_invaders_engine_glcd__AL
L.o Vspace_invaders_engine_glcd__ALL.cpp
ar -cr Vspace_invaders_engine_glcd__ALL.a Vspace_invaders_engine_glcd__ALL.o
ranlib Vspace_invaders_engine_glcd__ALL.a
sc_main.o sc_glcd128x64_TLM.o veril
ated.o Vspace_invaders_engine
lsystemc -lsystemc -o Vspace
make[1]: Leaving directory '/
l-DK/Projects/MyChip_Games/sp
dir'

=====
[SUCCESS] 아케이드 빌드가 성공적으로 완수되었습니다!
-> 실행 바이너리 실행 지점 명령: './obj_dir/Vspace_invaders_engine'
=====

./obj_dir/Vspace_invaders_engine_glcd

SystemC 3.0.2-Accellera --- Sep 16 2026 12:18:17
Copyright (c) 1996-2025 by all Contributors,
ALL RIGHTS RESERVED

```

Fig. 1. Q.10 빌드 성공과 시작 대기 화면의 체크무늬

플레이 화면에서는 외계인이 단순한 사각형이 아니라 8x8 비트맵의 고전 아케이드 형상으로 그려진다. 화면 상단에는 점수가 두 자리 숫자로 표시되고 우측 상단에는 남은 목숨이 대포 모양 아이콘으로 나타난다. Fig. 2에서 점수 05와 목숨 두 개를 확인할 수 있다.

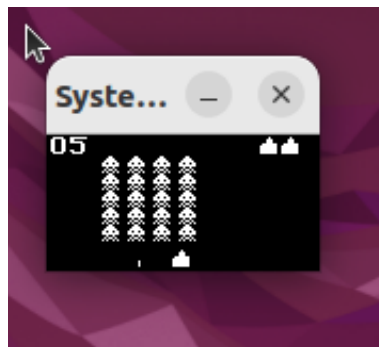


Fig. 2. Q.10 플레이 화면. 스프라이트로 그린 외계인 대형과 상단의 점수, 우측의 목숨 아이콘이 보인다

스프라이트 구현 방식 비교

같은 8x8 스프라이트인데 아두이노 C++와 베릴로그 RTL이 그것을 꺼내 쓰는 방식은 정반대다. 아두이노 코드(invaders_U8G2_IRQ.ino)에서 스프라이트는 프로그램 메모리에 저장된 데이터다. 프로세서가 P_TICK 인터럽트 핸들러 안에서 한 픽셀씩 좌표를 계산하고, 프레임버퍼 TableBMP의 해당 바이트를 찾아 비트를 쓴다. U8g2 라이브러리가 이 버퍼를 OLED로 옮긴다. 그림을 시간 순서대로 한 점씩 완성해 나가는 순차적인 방식이다.

베릴로그 RTL에서는 sprite_invader라는 8행의 8비트 레지스터 배열이 조합 논리 ROM으로 합성된다. 현재 주사 좌표에서 스프라이트 안의 오프셋을 역산해서, 예를 들면 sprite_invader(scan_y - curr_inv_y)[7 - (scan_x - curr_inv_x)]처럼, 해당 비트를 그 클럭에 바로 읽는다. 25마리 외계인에 대한 판정은 for 문이 전개된 병렬 비교 회로가 동시에 수행한다. 소프트웨어가 시간을 들여 순서대로 그리는 그림을 하드웨어는 공간을 들여 동시에 그린다.

실습 중 찾아낸 오류

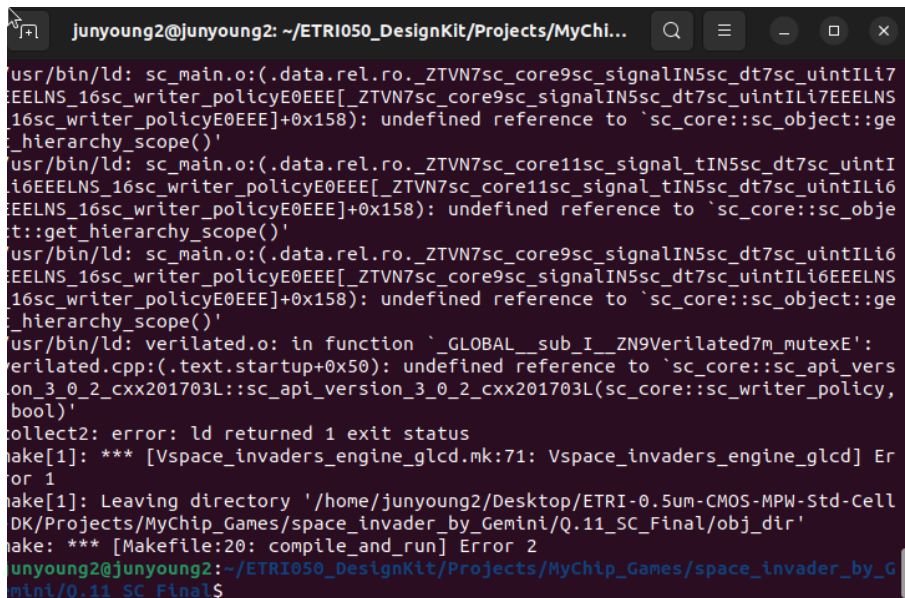
실습하면서 두 가지가 눈에 걸렸고, 코드를 읽어 원인을 확인했다. 첫째, 방향키를 누르면 대포가 부드럽게 움직이지 않고 순식간에 벽에 닿는다. 대포 이동 문장에 game_tick 조건이 없어서 50MHz 클럭마다 1픽셀씩 이동한다. 1초에 오천만 픽셀을 움직이는 셈이라 사람이 키를 누르는 시간과는 척도가 맞지 않는다.

둘째, 포탄을 쏘면 날아가는 과정이 보이지 않고 발사와 격추가 거의 동시에 일어난다. 포탄 이동도 클럭 1픽셀이라, 64픽셀 높이의 화면을 64클럭, 약 1.3마이크로초 만에 가로지른다. 디스플레이가 한 프레임을 표시하는 동안 발사부터 명중까지 다 끝나 버리니 화면에는 결과만 나타난다. 외계인이 순서대로 사라져야 재미가 있는 게임인데, 시간 척도가 어긋나니 그 재미가 사라진다.

코드를 더 읽어 보니 구조적인 문제도 두 개 있었다. 하나는 else if 사슬이다. Q.10 원본에서 대포 이동, 발사, 포탄 비행이 하나의 else if 사슬로 묶여 있어서, 키를 누르고 있는 동안에는 발사도 되지 않고 날아가던 포탄도 멈춘다. 다른 하나는 피격 처리다. 게임 로직 블록이 current_state에 STATE_DIE를 직접 대입하는데, FSM 블록도 같은 레지스터를 갱신하고 있어서 하나의 레지스터를 두 개의 always 블록이 구동하는 형태가 됐다. 시뮬레이션에서는 굴러가도 합성에서는 다중 구동 문제가 된다. 돌아가는 코드와 맞는 코드가 다르다는 것을 보여 주는 대목이다.

과제 3. Q.11 실습과 키보드 이상 반응의 원인

Q.11은 빌드부터 막혔다. 링커가 sc_core의 심볼을 찾지 못한다며 멈췄다(Fig. 3). 헤더는 /opt/systemc의 SystemC 3.0.2를 쓰는데 Makefile의 라이브러리 경로가 존재하지 않는 lib-linux64를 가리키고 있어서, 링커가 다른 버전의 라이브러리를 집어 온 것이다. 경로를 lib으로 고치고 LD_LIBRARY_PATH를 등록하니 실행됐다. 지난주 Makefile 과제와 마찬가지로 코드가 아니라 환경의 문제였다.



```
usr/bin/ld: sc_main.o: (.data.rel.ro._ZTVN7sc_core9sc_signalIN5sc_dt7sc_uintILi7EELNS_16sc_writer_policyE0EEE[_ZTVN7sc_core9sc_signalIN5sc_dt7sc_uintILi7EELNS_16sc_writer_policyE0EEE]+0x158): undefined reference to `sc_core::sc_object::get_hierarchy_scope()'
usr/bin/ld: sc_main.o: (.data.rel.ro._ZTVN7sc_core11sc_signal_tIN5sc_dt7sc_uintILi6EELNS_16sc_writer_policyE0EEE[_ZTVN7sc_core11sc_signal_tIN5sc_dt7sc_uintILi6EELNS_16sc_writer_policyE0EEE]+0x158): undefined reference to `sc_core::sc_object::get_hierarchy_scope()'
usr/bin/ld: sc_main.o: (.data.rel.ro._ZTVN7sc_core9sc_signalIN5sc_dt7sc_uintILi6EELNS_16sc_writer_policyE0EEE[_ZTVN7sc_core9sc_signalIN5sc_dt7sc_uintILi6EELNS_16sc_writer_policyE0EEE]+0x158): undefined reference to `sc_core::sc_object::get_hierarchy_scope()'
usr/bin/ld: verilated.o: in function `_GLOBAL__sub_I_ZN9Verilated7m_mutexE': verilated.cpp:(.text.startup+0x50): undefined reference to `sc_core::sc_api_version_3_0_2_cxx201703L:sc_api_version_3_0_2_cxx201703L(sc_core::sc_writer_policy, bool)'
collect2: error: ld returned 1 exit status
make[1]: *** [Vspace_invaders_engine_glcd.mk:71: Vspace_invaders_engine_glcd] Error 1
make[1]: Leaving directory '/home/junyoung2/Desktop/ETRI-0.5um-CMOS-MPW-Std-Cell/DK/Projects/MyChip_Games/space_invader_by_Gemini/Q.11_SC_Final/obj_dir'
make: *** [Makefile:20: compile_and_run] Error 2
junyoung2@junyoung2: ~/ETRI050_DesignKit/Projects/MyChip_Games/space_invader_by_Gemini/Q.11_SC_Final$
```

Fig. 3. Q.11 최초 빌드에서 멈춘 링크 오류

실행한 화면은 Fig. 4다. 게임은 돌아가지만 좌우 화살표를 누르면 대포가 순식간에 화면 끝까지 왔다 갔다 하고, 움직임도 이산적으로 된다.

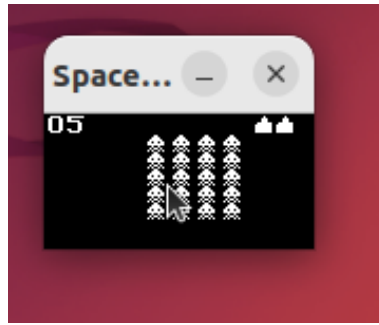


Fig. 4. Q.11 실행 화면. 방향키를 누르면 대포가 즉시 벽에 붙는다

원인은 테스트벤치와 RTL이 서로 다른 시간 척도로 동작하기 때문이다. SystemC 테스트벤치의 키보드 폴링 스레드는 16.6ms마다 한 번씩 키 상태를 읽어 버튼 신호에 기록하고, 다음 폴링까지 그 값을 유지한다. 시뮬레이션 시간으로 16.6ms는 50MHz 클럭 기준으로 약 83만 클럭이다. 그 사이 RTL은 매 클럭 대포를 1픽셀씩 움직인다. 키를 살짝 눌러도 하드웨어 입장에서는 수십만 클럭 동안 눌린 셈이니 대포가 그 즉시 벽에 붙는다. 입력은 프레임 속도로 들어오는데 동작은 클럭 속도로 처리하는, 시간 관리의 실패다.

과제 4. Q.13 실습과 테스트벤치, RTL의 변경 사항

Q.13의 시뮬레이션 디렉토리에는 RTL이 다섯 종류로 준비되어 있다. make를 실행하면 어느 RTL을 쓸지 번호로 선택하게 되어 있다(Fig. 5). 이름만 봐도 0번이 원본이고 4번이 최종 수정본임을 알 수 있다.

```

junyoung2@junyoung2: ~/ETRI050_DesignKit/Projects/MyChi...
*****
!!! space_invaders_engine_glcd.v not exist in the RTL directory !!!
*****
Try one of following RTLs,
0_space_invaders_engine_glcd.v.Org_wild_cannon
1_space_invaders_engine_glcd.v.bullet_no_disp
2_space_invaders_engine_glcd.v.btn_fire
3_space_invaders_engine_glcd.v.FPGA_err
4_space_invaders_engine_glcd.v.Ok
Choose [0/1/2/3/4]: 0
make: *** [Makefile:59: all] Error 1
junyoung2@junyoung2:~/ETRI050_DesignKit/Projects/MyChip_Games/space_invader_by_G
mini/Q.13_SC_Final_PSCE/simulation$ make run
verilator --sc -Wno-WIDTHTRUNC -Wno-WIDTHEXPAND -Wno-UNSIGNED --trace --timing -
-pins-sc-uint \
    --top-module space_invaders_engine_glcd --exe --build \
    -CFLAGS -g -CFLAGS -I../c_untimed -CFLAGS -I/opt/syst
emc/include -CFLAGS -DVCD_TRACE_TEST_TB -CFLAGS -DVCD_TRACE_DUT_VERILOG -CFLAGS
-DVCD_TRACE_GLCD -LDFLAGS -lm -LDFLAGS -lgs -LDFLAGS -lSDL2 ../space_invaders_e
ngine_glcd/space_invaders_engine_glcd.v ./sc_main.cpp ./sc_glcd128x64_TLM.cpp ./
sc_space_invaders_engine_glcd_TB.cpp
%Error: Unknown warning specified: -Wno-WIDTHTRUNC
make: *** [Makefile:106: obj_dir/Vspace_invaders_engine_glcd] Error 1

```

Fig. 5. Q.13 시뮬레이션 디렉토리에서 RTL 다섯 종류를 고르는 메뉴

빌드에서는 구버전 Verilator가 최신 경고 옵션과 --timing 플래그를 받지 못하는 문제, 그리고 Verilator의 VCD 추적 코드가 SystemC 3.0.2의 바뀐 내부 API와 충돌하는 문제가 차례로 나타났다. 파형 추적은 이번 비교에 필요하지 않아서 끄고 진행했다. 실행하니 터미널에 프레임 카운터가 찍히고 게임이 정상적으로 진행됐다(Fig. 6).

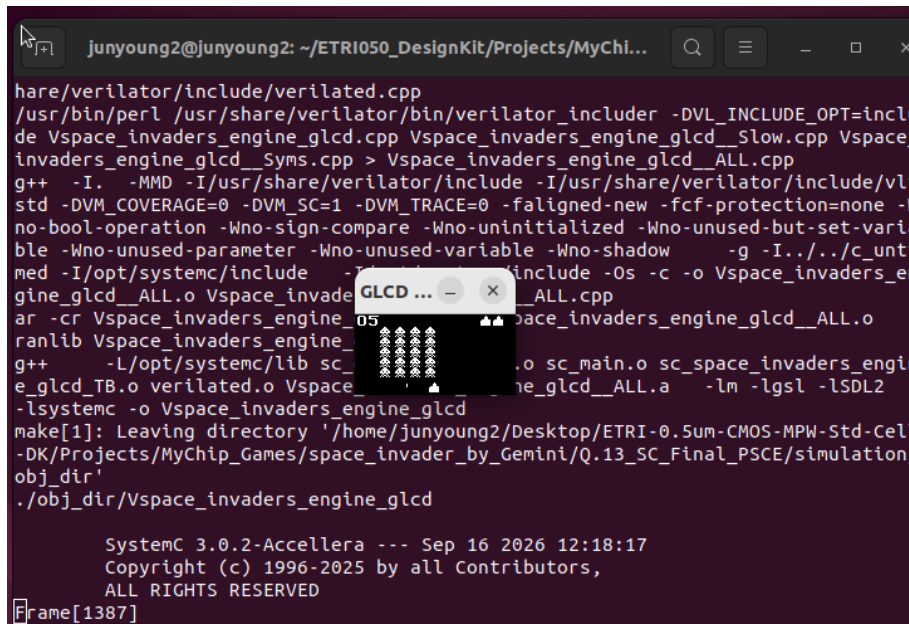


Fig. 6. Q.13 실행 화면과 터미널의 프레임 카운터

테스트벤치의 변경

0번 원본은 Q.11의 RTL과 사실상 같은 코드다. 그런데 Q.13 환경에서 실행하니 조작감이 Q.11보다 훨씬 자연스러웠다. 같은 RTL이 다르게 느껴진다면 차이는 테스트벤치에서 온다. 실제로 테스트벤치가 크게 바뀌어 있었다.

Q.11의 테스트벤치는 클럭을 만드는 스레드와 키보드를 폴링하는 스레드를 직접 갖고 있었고, sc_main이 테스트벤치와 엔진과 디스플레이를 각각 만들어 연결했다. Q.13에서는 테스트벤치가 DUT과 GLCD 디스플레이 모델을 내부에 품는 구조로 바뀌었고, sc_main은 테스트벤치 하나만 세운다. 클럭은 sc_clock으로 대체됐다. 키보드 처리는 디스플레이 모델 안의 Button_Thread로 옮겨갔는데, 16.6ms 간격으로 키 레벨을 훑는 폴링 대신 SDL의 KEYDOWN과 KEYUP 이벤트로 버튼 신호를 기록한다. 화면을 띄우는 장치 모델이 입력 장치까지 맡게 된 셈이다. EMULATED_CO_SIM 매크로 분기도 새로 생겼다. 같은 테스트벤치에 Verilator가 만든 시뮬레이션 모델 대신 FPGA 에뮬레이션 모델을 꽂을 수 있게 만든 것인데, 다음 주에 예고된 코에뮬레이션을 위한 준비다.

RTL의 변경

키보드 입력을 보완하기 위해 RTL이 어떻게 바뀌었는지 0번과 4번을 diff로 비교했다. 변경은 네 가지다.

첫째, FSM에 STATE_CHK_BTN 상태가 추가됐다. 한 프레임의 주사가 끝나는 지점에서 이 상태를 거치며 chk_btn 플래그가 한 번 세팅되고, 대포 이동은 chk_btn이 세팅된 그 클럭에만 1픽셀씩 일어난다. 이동이 클럭 단위에서 프레임 단위로 바뀐 것이다.

둘째, btn_fire를 한 클럭 지연시킨 신호와 조합해서 상승 에지 검출 btn_fire_pe를 만들었다. 발사가 버튼의 레벨이 아니라 눌리는 사건으로 처리된다.

셋째, 피격 시 current_state에 직접 쓰던 코드가 player_hit 플래그를 세우는 방식으로 바뀌었다. 상태 천이는 FSM 블록이 next_state로만 결정하므로 다중 구동 문제가 해소됐다.

넷째, 포탄 비행에도 game_tick이 걸렸고, 상태 인코딩은 원-핫으로 바뀌었다. game_tick 값은 833333에서 16000으로 줄었는데, 에뮬레이션 환경의 낮은 클럭 주파수에 60Hz를 맞추기 위한 조정이다. 이 네 가지는 모두 같은 주제의 변주다. 사람의 입력 시간과 디스플레이의 프레임

시간에 하드웨어의 클럭 시간을 맞추는 일이다.

소프트웨어 언어와 하드웨어 언어

이번 실습을 거치면서 둘의 차이를 내 문장으로 정리해 봤다. 합성 가능성부터다. 하드웨어 언어로 쓴 알고리즘이 실제 회로가 되려면 RTL, 즉 레지스터와 레지스터 사이의 전송을 클럭에 맞춰 기술하는 수준에 머물러야 한다. always 문이 절차서가 아니라 회로도로 읽혀야 한다.

두 번째는 비트 단위의 객체다. C의 int와 달리 reg [6:0]처럼 폭을 적는 순간 그 폭이 곧 하드웨어의 크기다. 지난주의 포트 타입 불일치 에러도 이 차이에서 왔다. 세 번째는 다중치 논리다. 0과 1 외에 미지의 X와 고임피던스 Z가 있다. 초기화하지 않은 레지스터가 X로 드러나는 것은 소프트웨어에는 없는 현상이다.

네 번째는 병렬성과 시간이다. 하드웨어의 문장들은 순서대로 실행되지 않고 동시에 평가된다. 그래서 시간은 문장의 순서가 아니라 클럭 에지와 사건으로 관리해야 하고, 상태는 레지스터에 저장하며 사건은 에지로 포착한다. Q.11의 키보드 이상 반응이 시간 관리의 실패 사례고, Q.13의 btn_fire_pe가 사건 관리의 사례다.

SystemC에 대한 이해

SystemC는 C++ 위에 하드웨어의 개념을 엮은 클래스 라이브러리와, 그것을 구동하는 이벤트 기반 병렬 시뮬레이션 엔진이다. 모듈과 포트와 신호라는 하드웨어 개념을 C++의 클래스로 표현하고, SC_THREAD와 wait()로 프로세스와 시간의 경과를 기술한다. 앞서 정리한 하드웨어의 개념들, 비트 폭과 병렬성과 시간 관리와 사건을 C++ 안에 가져오는 층이다.

Verilator가 베릴로그 RTL을 SystemC 모듈로 변환해 주기 때문에, 검증 대상 하드웨어와 C++로 만든 주변 장치 모델을 한 프로세스 안에서 함께 돌릴 수 있다. 이번 실습의 가상 LCD와 키보드가 그 주변 장치였다. 실물 기관 없이 디스플레이와 입력 장치까지 갖춘 검증 환경을 코드로 만들 수 있다는 것, 그리고 그 환경 전체가 다시 읽고 고칠 수 있는 코드라는 것이 SystemC의 장점이라고 정리한다.

과제를 마치고

지난주에 검증하고 정리하는 단계에서 엔지니어링이 일어난다고 썼다. 이번 주가 그 경우였다. 인공지능이 만들어 준 코드는 컴파일됐고 화면도 나왔지만, 시간 척도가 어긋난 결함이 안에 숨어 있었다. 게임이 돌아간다는 사실과 설계가 올바르다는 사실은 다르다. 실행해 보고, 이상한 점을 관찰하고, diff로 원인을 특정하는 일은 사람이 했다. 인공지능이 코드의 대부분을 써 주는 시대에도, 그 코드가 맞는지 확인하는 능력만큼은 사람이 갖고 있어야 한다는 생각이 다시 굳어졌다.